Course Outline: Frontend Optimization Strategies

Title: Frontend Optimization Strategies **Skill:** Skill 35: Debugging and optimizing the frontend **Learnpack:** + Estrategias de optimización del frontend **File:** s35-w12d36-estrategias-de-optimizacion-del-frontend **Prerequisite:** s35-w12d36-midiendo-el-desempeno-del-frontend-core-web-vitals **Target Audience:** Beginner developers who can already measure frontend performance and want to learn how to fix it **Total Lessons:** 15 **Format:** Mixed (20% hands-on = 3 lessons)

Course Description

You know how to measure your app's performance — now it's time to improve it. This course covers the core strategies for making React applications fast: splitting code so the browser loads only what it needs, memoizing values and components so React skips redundant work, extracting reusable logic into custom hooks so your codebase stays clean and composable, and understanding the principle that the fastest work is the work you never have to do. Each strategy builds on the last, giving you a practical toolkit for delivering snappy, efficient user experiences.

Course Philosophy

This course emphasizes:

- Practical optimization over premature abstraction — optimize what you can measure
- Layered understanding — each strategy addresses a different kind of waste (load, compute, fetch)
- The path of least resistance — start with the simplest optimization that makes a difference

---

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Code Splitting	3
02 - Memoization	5
03 - Reuse Patterns	4
04 - Assessment	1
05 - Outro	1
Total	**15**

00.0 Fast by Design 🕮

Learning Objectives:

- Recognize the relationship between bundle size, re-renders, and perceived performance
- Understand how the three optimization strategies in this course (code splitting, memoization, reuse patterns) each address a distinct performance bottleneck
- Connect this course to the previous one: from measuring with Core Web Vitals to fixing the underlying causes

Content Outline:

- The gap between measuring and fixing: what Core Web Vitals tell you, and what they don't tell you about *why*
- Three categories of frontend waste: unnecessary loading, unnecessary computation, unnecessary re-fetching
- A map of the course: Code Splitting → Memoization → Reuse Patterns → Cache Memory

Transitions: Students already understand what slow looks like via Core Web Vitals. This lesson frames *why* apps get slow — and previews the toolkit we'll build to fix it. Section 01 starts with the first strategy: loading less.

Key Principles:

- Optimization is targeted, not blanket — fix the specific bottleneck you've measured
- Fast apps are designed, not retrofitted — the earlier you apply these strategies, the less costly they are
- The three strategies are complementary: splitting loads less, memoization computes less, cache memory fetches less

Examples:

- A React app that imports all routes upfront vs. one that loads them on demand — same code, very different Time-to-Interactive
- A component that re-renders on every parent update vs. one wrapped in `React.memo` — same output, very different CPU usage

---

01.0 Code Splitting 🕮

Learning Objectives:

- Explain what code splitting is and why it improves load performance
- Describe how JavaScript bundlers combine files and why bundle size matters
- Understand the trade-off between a single bundle and multiple smaller chunks

Content Outline:

- What is a bundle, and why does it grow over time?
- The user pays for every byte upfront in a non-split app
- Code splitting: the idea of delivering only the code the current route or interaction needs
- How modern bundlers (Webpack, Vite) implement splitting with dynamic imports

- What this section covers: the concept (01.0), the React implementation (01.1), and a practice exercise (01.2)

Transitions: This lesson establishes *what* code splitting is and *why* it matters. The next lesson covers the React-specific tools — `React.lazy` and `Suspense` — that make it practical.

Key Principles:

- Bundle size directly affects Time-to-Interactive (LCP and FID)
- Splitting is most effective at route boundaries — users who never visit a route shouldn't pay for its code
- Dynamic imports are the mechanism; `React.lazy` is the React abstraction over them

Examples:

- A monolithic bundle of 2MB vs. an initial bundle of 200KB + on-demand chunks — the latter loads 10× faster on a 3G connection
- `import('./HeavyComponent')` — a dynamic import that the bundler turns into a separate chunk

---

01.1 Lazy Loading with React 🕮

Learning Objectives:

- Implement route-level code splitting using `React.lazy` and `Suspense`
- Apply component-level splitting for conditionally rendered heavy components
- Handle the loading state while a chunk is being fetched

Content Outline:

- `React.lazy`: wrapping a dynamic import so React handles the async loading
- `Suspense`: the boundary that renders a fallback while the lazy component loads
- Route-level splitting — the most impactful place to split (one route = one chunk)
- Component-level splitting — for modals, drawers, and anything not rendered on initial load
- Error boundaries alongside `Suspense` — what happens when a chunk fails to load

Transitions: Now you know the theory and the API. The next lesson is a hands-on exercise where you'll apply lazy loading to a real route. Then Section 02 shifts to a different bottleneck: unnecessary re-renders.

Key Principles:

- `React.lazy` only works with default exports — named exports must be re-wrapped
- The `Suspense` fallback should be meaningful — a skeleton or spinner, not a blank screen
- Splitting a route that's always visited immediately has no benefit — split what users reach later or conditionally

Examples:

```
// Before: imported eagerly — always in the bundle
import Dashboard from './Dashboard';

// After: imported lazily — loaded only when this route is visited
const Dashboard = React.lazy(() => import('./Dashboard'));

function App() {
  return (
    <Suspense fallback={<Spinner />}>
      <Routes>
        <Route path="/dashboard" element={<Dashboard />} />
      </Routes>
    </Suspense>
  );
}
```

01.2 Split a Route 💻

Challenge Prompt: Refactor the provided React app so that the `Settings` and `Analytics` pages are lazy-loaded using `React.lazy` and `Suspense`, with a loading spinner shown while each page loads.

Solution Code:

```
import React, { Suspense } from 'react';
import { Routes, Route } from 'react-router-dom';
import Spinner from './Spinner';

const Settings  = React.lazy(() => import('./pages/Settings'));
const Analytics = React.lazy(() => import('./pages/Analytics'));

export default function App() {
  return (
    <Suspense fallback={<Spinner />}>
      <Routes>
        <Route path="/settings"  element={<Settings />} />
        <Route path="/analytics" element={<Analytics />} />
      </Routes>
    </Suspense>
  );
}
```

Challenge Code:

```
import React from 'react';
import { Routes, Route } from 'react-router-dom';
import Spinner from './Spinner';
```

```
// TODO: import Settings and Analytics lazily using React.lazy
import Settings  from './pages/Settings';
import Analytics from './pages/Analytics';

export default function App() {
  return (
    // TODO: wrap Routes in a Suspense boundary with <Spinner /> as fallback
    <Routes>
      <Route path="/settings"  element={<Settings />} />
      <Route path="/analytics" element={<Analytics />} />
    </Routes>
  );
}
```

02.0 Memoization 🕮

Learning Objectives:

- Define memoization and explain how it reduces redundant computation
- Describe how the Single Responsibility Principle (SRP) enables effective memoization
- Identify the three memoization tools covered in this section and when each applies

Content Outline:

- What is memoization? Caching a computed result so it's not recalculated when the inputs haven't changed
- The React rendering model: why components re-render by default whenever a parent does
- How re-renders waste CPU: every re-render calls the component function, diffs the output, and may update the DOM
- The Single Responsibility Principle (SRP) as a prerequisite: a component that does one thing is easy to memoize because its inputs are clear and minimal
- Three tools this section covers: `React.memo` (memoize a component), `useMemo` (memoize a value), and Cache Memory (memoize a network request)
- Anti-pattern preview: memoizing everything is not the goal — measure first

Transitions: This lesson sets the stage. 02.1 and 02.2 cover the two React hooks in detail. 02.3 closes with the broader principle of cache memory. The section ends with a practice challenge in 02.4.

Key Principles:

- Memoization trades memory for CPU time — it stores a previous result to skip recomputation
- React re-renders are not inherently bad; *unnecessary* re-renders are
- SRP makes memoization effective: a component with 10 responsibilities has 10 reasons to re-render; one with 1 responsibility re-renders only when that one thing changes
- Don't memoize what's cheap to recompute — the overhead of memoization must be worth the savings

Anti-pattern (woven in): Wrapping every component in `React.memo` by default. This adds overhead without benefit for components that *should* re-render frequently. Profile first, then memoize.

Examples:

- A `<UserList>` that re-renders every time its parent's theme changes, even though the user data didn't change — a candidate for `React.memo`
- A filtered and sorted list recalculated on every keystroke — a candidate for `useMemo`

02.1 React.memo 🕮

Learning Objectives:

- Wrap a component in `React.memo` to prevent re-renders when props haven't changed
- Explain how React's shallow prop comparison works and when it falls short
- Decide when `React.memo` is worth adding

Content Outline:

- How `React.memo` works: wraps a component and performs a shallow comparison of its props before deciding to re-render
- Shallow comparison: primitive values are compared by value; objects and arrays are compared by reference
- The reference trap: a new object `{}` or array `[]` created inline in JSX always fails the shallow comparison — `React.memo` has no effect
- Custom comparison function: the second argument to `React.memo` for cases where shallow comparison isn't enough
- When `React.memo` is worth it: components that render frequently, receive the same props most of the time, and have non-trivial render cost

Transitions: `React.memo` prevents a component from re-rendering. The next lesson covers `useMemo`, which prevents a *value* from being recomputed. Together they cover both levels: component and computation.

Key Principles:

- `React.memo` is a higher-order component (HOC), not a hook — it wraps the component definition
- The reference trap is the most common reason `React.memo` appears to do nothing
- Profiling in React DevTools before and after is the only reliable way to confirm a benefit

Examples:

```
// Without memo: re-renders every time the parent renders
function ExpensiveList({ items }) {
  return <ul>{items.map(i => <li key={i.id}>{i.name}</li>)}</ul>;
}
```

```
// With memo: re-renders only when `items` reference changes
const ExpensiveList = React.memo(function ExpensiveList({ items }) {
  return <ul>{items.map(i => <li key={i.id}>{i.name}</li>)}</ul>;
});
```

02.2 useMemo 🕮

Learning Objectives:

- Use `useMemo` to memoize an expensive computation across renders
- Correctly specify the dependency array to control when the value is recomputed
- Distinguish between cases where `useMemo` helps and where it adds unnecessary complexity

Content Outline:

- What `useMemo` does: runs a function once, caches the result, and only reruns it when a listed dependency changes
- Syntax: `const result = useMemo(() => expensiveCalc(a, b), [a, b])`
- The dependency array: same rules as `useEffect` — list every value from the component scope that the function reads
- Common use cases: expensive filtering/sorting/formatting, derived data that would be recalculated on every render
- `useMemo` vs `useCallback`: `useMemo` memoizes a *value*; `useCallback` memoizes a *function reference* (useful for passing stable callbacks to memoized children)
- The stale closure trap: omitting a dependency means the cached value will reference the old version of that variable

Transitions: `useMemo` handles expensive calculations; `React.memo` handles component re-renders. 02.3 closes the section with a related idea: cache memory, which applies the same principle at the network level. Then 02.4 gives you hands-on practice.

Key Principles:

- Only memoize values that are genuinely expensive to compute — for simple transforms, the overhead of `useMemo` costs more than it saves
- Every value used inside the function that changes over time must appear in the dependency array
- `useMemo` does not prevent side effects — it only caches return values of pure computations

Examples:

```
function ProductList({ products, filter }) {
  // Recomputed only when products or filter changes, not on every render
  const filtered = useMemo(
    () => products.filter(p => p.category === filter),
    [products, filter]
  );
```

```
  return <ul>{filtered.map(p => <li key={p.id}>{p.name}</li>)}</ul>;
}
```

02.3 Cache Memory 🕮

Learning Objectives:

- Explain the principle "the fastest request is the one you don't make"
- Identify common cases where frontend apps make redundant network requests
- Apply basic in-memory caching patterns to avoid re-fetching data that hasn't changed

Content Outline:

- The performance cost of a network request: latency + serialization + state updates + re-renders
- The principle: if data hasn't changed, don't re-fetch it — serve it from memory
- In-component caching: holding data in state and not re-fetching unless a dependency actually changes
- `useMemo` applied to derived request parameters: avoid constructing a new request object on every render
- Stale-while-revalidate: serving cached data immediately while a fresh fetch runs in the background — a pattern popularized by libraries like React Query and SWR
- What this is NOT: server-side caching, Redis, or distributed cache — those are covered in a later skill on backend caching

Transitions: This closes Section 02. The memoization toolkit now spans three levels: component re-renders (`React.memo`), computed values (`useMemo`), and network requests (cache memory). Section 03 shifts to a different kind of optimization: how to structure code so that logic is reusable without being duplicated.

Key Principles:

- Fetching is expensive at every layer: network latency, parsing, React state updates, re-renders
- Cache invalidation is the hard part — decide upfront what makes your cached data stale
- Simple in-memory caches (a `useRef` or module-level object) are often enough for small apps; React Query / SWR scale this pattern

Examples:

```
// Without caching: fetches on every render where userId is in the dependency
list
useEffect(() => {
  fetch(`/api/users/${userId}`).then(r => r.json()).then(setUser);
}, [userId]);

// With simple cache: skips the fetch if we already have the user
const cache = {};
useEffect(() => {
```

```
  if (cache[userId]) { setUser(cache[userId]); return; }
  fetch(`/api/users/${userId}`)
    .then(r => r.json())
    .then(data => { cache[userId] = data; setUser(data); });
}, [userId]);
```

02.4 Memoize a Component 💻

Challenge Prompt: Wrap the `ProductList` component in `React.memo` and use `useMemo` inside `FilteredResults` to prevent recomputing the filtered list on every render when only unrelated state changes.

Solution Code:

```
import React, { useState, useMemo } from 'react';

const ProductList = React.memo(function ProductList({ products }) {
  return <ul>{products.map(p => <li key={p.id}>{p.name}</li>)}</ul>;
});

function FilteredResults({ allProducts }) {
  const [category, setCategory] = useState('all');
  const [count, setCount]       = useState(0);

  const filtered = useMemo(
    () => category === 'all' ? allProducts : allProducts.filter(p =>
p.category === category),
    [allProducts, category]
  );

  return (
    <div>
      <button onClick={() => setCount(c => c + 1)}>Clicked {count}
times</button>
      <select onChange={e => setCategory(e.target.value)}>
        <option value="all">All</option>
        <option value="books">Books</option>
      </select>
      <ProductList products={filtered} />
    </div>
  );
}

export default FilteredResults;
```

Challenge Code:

```jsx
import React, { useState, useMemo } from 'react';

// TODO: wrap ProductList with React.memo
function ProductList({ products }) {
  return <ul>{products.map(p => <li key={p.id}>{p.name}</li>)}</ul>;
}

function FilteredResults({ allProducts }) {
  const [category, setCategory] = useState('all');
  const [count, setCount]       = useState(0);

  // TODO: memoize this computation with useMemo
  const filtered = category === 'all' ? allProducts : allProducts.filter(p =>
p.category === category);

  return (
    <div>
      <button onClick={() => setCount(c => c + 1)}>Clicked {count}
times</button>
      <select onChange={e => setCategory(e.target.value)}>
        <option value="all">All</option>
        <option value="books">Books</option>
      </select>
      <ProductList products={filtered} />
    </div>
  );
}

export default FilteredResults;
```

03.0 Reuse Patterns 🕮

Learning Objectives:

- Describe the three main patterns for reusing component logic in React: Custom Hooks, Higher-Order Components (HOCs), and Render Props
- Explain why duplicated logic is a performance and maintainability problem
- Identify which pattern to reach for in a given situation

Content Outline:

- The reuse problem: when two components share behavior, where does that behavior live?
- Three solutions React developers have developed over time: HOCs, Render Props, and Custom Hooks
- A brief history of why HOCs and Render Props existed before hooks, and why Custom Hooks are now preferred
- Decision guide: Custom Hooks for shared stateful logic, HOCs when wrapping is needed (e.g., logging, auth), Render Props when you need to inject rendering behavior

- What this section covers: HOCs and Render Props in context (03.1), Custom Hooks in depth (03.2), hands-on extraction (03.3)

Transitions: The previous section covered how to avoid redundant computation inside components. This section covers how to avoid duplicating the component logic itself. 03.1 puts HOCs and Render Props in historical context. 03.2 covers Custom Hooks, the modern default.

Key Principles:

- DRY (Don't Repeat Yourself) applies to logic, not just data — shared behavior should live in one place
- Custom Hooks are the default choice in modern React: they compose cleanly, they're easy to test, and they don't add wrapper nodes to the component tree
- Performance benefit: a shared hook encapsulates its own memoization — you fix the performance in one place, and every consumer benefits

Examples:

- A `useFetch` hook shared across 5 components vs. the same `useEffect`/`useState` pattern copied into all 5
- A `withAuth` HOC that checks authentication before rendering any page component

---

03.1 HOCs and Render Props 🕮

Learning Objectives:

- Explain what a Higher-Order Component is and how it wraps a component to inject behavior
- Describe the Render Props pattern and how it passes rendering control to the consumer
- Recognize when these patterns still make sense in a modern React codebase

Content Outline:

- Higher-Order Components (HOCs): a function that takes a component and returns an enhanced component
  - Classic use cases: authentication gates, analytics wrappers, error boundaries as HOCs
  - The wrapper hell problem: nesting 5 HOCs produces 5 extra nodes in the tree and makes debugging harder
  - Still relevant for: third-party library integration, code that must work without hooks (class components)
- Render Props: a component that receives a function as a prop and calls it to render its output
  - Classic use case: mouse position tracker, data fetchers that pass data down via a render prop
  - Replaced by hooks in most cases but still useful for libraries that expose this pattern
- Anti-pattern: using HOCs or Render Props when a Custom Hook would be simpler and flatter

Transitions: HOCs and Render Props solved real problems, but they introduced complexity. Custom Hooks solve the same problems without the wrapper overhead. 03.2 covers Custom Hooks as the modern replacement.

Key Principles:

- HOCs compose vertically (wrapper on wrapper); Custom Hooks compose horizontally (hook calling hook)
- Render Props are powerful but verbose — their use has largely been replaced by hooks that return values directly
- You will encounter HOCs and Render Props in real codebases and third-party libraries — understanding them is essential even if you don't write them daily

Examples:

```
// HOC: inject current user into any component
function withUser(Component) {
  return function WrappedComponent(props) {
    const user = useCurrentUser();
    return <Component {...props} user={user} />;
  };
}

// Render Prop: passes mouse position to whatever the caller wants to render
function MouseTracker({ render }) {
  const [pos, setPos] = useState({ x: 0, y: 0 });
  return (
    <div onMouseMove={e => setPos({ x: e.clientX, y: e.clientY })}>
      {render(pos)}
    </div>
  );
}
```

03.2 Custom Hooks 🕮

Learning Objectives:

- Build a custom hook that encapsulates stateful logic and returns a clean interface
- Compose custom hooks from other hooks (including other custom hooks)
- Identify logic worth extracting into a custom hook

Content Outline:

- What is a custom hook? Any function that starts with `use` and calls other hooks
- Rules of Hooks apply: custom hooks follow the same rules as built-in hooks — only call at the top level, only call from React functions
- The extraction signal: when you copy the same `useState` + `useEffect` pattern into a second component, extract it
- Anatomy of a custom hook: receives parameters, manages state/effects internally, returns values and/or handlers

- Composing custom hooks: a `usePaginatedSearch` hook that calls `useFetch` and `useDebounce` internally
- Testing advantage: custom hooks can be tested in isolation with `@testing-library/react-hooks`

Transitions: Now you know how to build custom hooks. 03.3 is a hands-on exercise where you'll extract duplicated logic from two components into a single reusable hook.

Key Principles:

- The naming convention `use*` is enforced by the React linter — it signals that the function follows hook rules
- A good custom hook hides complexity and exposes a simple interface — the consumer shouldn't need to know how it works internally
- Custom hooks are the primary mechanism for sharing and reusing logic in modern React

Examples:

```
// Custom hook: encapsulates local storage sync
function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    try { return JSON.parse(localStorage.getItem(key)) ?? initialValue; }
    catch { return initialValue; }
  });

  const set = (next) => {
    setValue(next);
    localStorage.setItem(key, JSON.stringify(next));
  };

  return [value, set];
}

// Usage
function ThemeToggle() {
  const [theme, setTheme] = useLocalStorage('theme', 'light');
  return <button onClick={() => setTheme(theme === 'light' ? 'dark' :
'light')}>{theme}</button>;
}
```

03.3 Build a Custom Hook 💻

Challenge Prompt: Extract the duplicated data-fetching logic from `UserProfile` and `ProductDetail` into a single `useFetch` custom hook that accepts a URL and returns `{ data, loading, error }`.

Solution Code:

```
import { useState, useEffect } from 'react';

function useFetch(url) {
  const [data,    setData]    = useState(null);
  const [loading, setLoading] = useState(true);
  const [error,   setError]   = useState(null);

  useEffect(() => {
    let cancelled = false;
    setLoading(true);
    fetch(url)
      .then(r => r.json())
      .then(d => { if (!cancelled) { setData(d); setLoading(false); } })
      .catch(e => { if (!cancelled) { setError(e); setLoading(false); } });
    return () => { cancelled = true; };
  }, [url]);

  return { data, loading, error };
}

export default useFetch;
```

Challenge Code:

```
import { useState, useEffect } from 'react';

// TODO: implement useFetch(url) that returns { data, loading, error }
function useFetch(url) {
  // your code here
}

export default useFetch;
```

04.0 Frontend Optimization 🫣

Learning Objectives:

- Demonstrate understanding of code splitting with `React.lazy` and `Suspense`
- Apply knowledge of `React.memo` and `useMemo` to identify and fix unnecessary re-renders
- Distinguish between Custom Hooks, HOCs, and Render Props and select the right pattern for a given scenario
- Recall the cache memory principle and its application to frontend request optimization

Question Format: Scenario-based (multiple choice)

Questions:

1. A React app has a single bundle of 3.4MB. Half of the code belongs to an admin dashboard that only 5% of users ever visit. What is the most effective first optimization?

    - A) Compress all images on the dashboard
    - B) Split the dashboard into a separate chunk using `React.lazy` ☑
    - C) Add `React.memo` to every component on the dashboard
    - D) Move the dashboard to a different server

2. You wrap a `<Chart>` component with `React.memo`. After testing, you notice it still re-renders every time the parent renders. The parent passes `filters` as a prop, defined as `const filters = { type: 'bar' }` inline in JSX. What is causing the issue?

    - A) `React.memo` doesn't work with object props
    - B) `Chart` has too many props for shallow comparison to work
    - C) A new `filters` object is created on every parent render, so the reference always changes ☑
    - D) `React.memo` requires a custom comparison function for all cases

3. A component calculates a sorted and filtered list from 10,000 items on every render. The list only changes when the `filter` or `items` props change. Which hook should you use to optimize this?

    - A) `useCallback`
    - B) `useRef`
    - C) `useMemo` ☑
    - D) `useReducer`

4. You have a `useWindowSize` custom hook and a `withWindowSize` HOC that do exactly the same thing. A new component needs this behavior. Which should you use?

    - A) The HOC, because it has been around longer and is more battle-tested
    - B) The custom hook, because it composes cleanly without adding wrapper nodes ☑
    - C) Render Props, because they're more explicit
    - D) Neither — inline the logic directly

5. An app fetches the same `/api/config` endpoint in 8 different components. The config never changes during a session. What is the best optimization?

    - A) Add a `Suspense` boundary around each component
    - B) Use `React.memo` on each component that calls the endpoint
    - C) Fetch once and store the result in memory; serve subsequent reads from cache ☑
    - D) Move the fetch to a `useEffect` with an empty dependency array in each component

6. A `Suspense` boundary is set up with `fallback={<Spinner />}`. A lazy-loaded component takes 2 seconds to load. What does the user see during that time?

    - A) A blank white screen
    - B) The `<Spinner />` component ☑
    - C) The previous page
    - D) An error message

7. Which statement about `useMemo` is correct?
    - A) It prevents the component from re-rendering
    - B) It caches the result of a function and returns it until a listed dependency changes ☑
    - C) It replaces `useState` for expensive state initialization
    - D) It works the same as `useCallback`

8. You need to inject error tracking into every page component without modifying each one individually. Which pattern is most appropriate?
    - A) Custom Hook
    - B) Render Props
    - C) Higher-Order Component ☑
    - D) `useMemo`

Evaluation Criteria:

- 8/8: Excellent mastery of all optimization strategies
- 6–7/8: Strong understanding with minor gaps — review the missed topics
- 4–5/8: Partial understanding — revisit the sections covering missed questions
- Below 4: Review the full course before proceeding

Score Interpretation: A passing score is 6/8. If you scored below 6, return to the relevant sections before moving on to the next skill.

05.0 Conclusion

Content Outline:

- Course recap: what students accomplished
    - Learned to split code so the browser loads only what it needs
    - Prevented unnecessary re-renders with `React.memo` and `useMemo`
    - Applied the cache memory principle to avoid redundant network requests
    - Extracted reusable logic into custom hooks and understood HOCs and Render Props
- Connection to bigger picture
    - These strategies work together: a well-split, well-memoized, well-structured app is the foundation for everything that comes next
    - The next skills cover serialization and backend caching — optimization continues on the server side
- Next steps and continued learning
    - Explore React DevTools Profiler to identify real-world re-render bottlenecks
    - Investigate React Query or SWR for production-grade server-state caching
    - Read the React docs on `useCallback` as the counterpart to `useMemo` for function references
- Resources for further exploration
    - React Docs: `React.lazy`, `React.memo`, `useMemo`, Custom Hooks
    - Web.dev: Code splitting guide
    - React Query documentation

- Encouragement and celebration
    - You now have a practical toolkit for building fast React applications. Performance is a feature — and you know how to ship it.

Note: This is conclusion only - no theory, exercises, or assessment.